Road-Attribute Editor — Product & Research Plan

> **Owner:** Tuệ (product lead) · **Updated:** 2026-06-22
>
> **Target:** maxspeed on most of the national + expressway network (~28,000 km) at **launch, early September 2026**.
>
> Productizes `traffic/lanes/` into TASCO's internal multi-source road-attribute editor. Ladders to
>
> Enterprise Architecture ("Editor Spatial DB Server") and the O3
>
> enrichment workstream. Extends the 2026-06-15 scope pivot (road-by-road, evidence-per-edit).

---

1. TL;DR

We are not "editing 28,000 km of road." We are **solving one estimation problem** — *what is the speed limit on each stretch of road, and how sure are we?* — and then spending scarce editor-hours only where the math says a human is needed. Framed that way, an early-September launch is feasible: the busiest national roads are exactly the ones where the estimate is most confident and cheapest to confirm.

This document opens with the **plain five-step flow** (§2), **what we're building and how it grows** beyond maxspeed (§3), and **the team/roles it takes** (§4). The deeper half then states the estimation problem as **one objective**, decomposes it into **eight sub-problems** (each with its method, the research that backs it, and its status), and lays out the **phased plan** to ship it.

> **Launch constraint (drives everything below).** ~10 weeks to early September. **Purchased + CV-processed
>
> satellite cannot make that date** (procurement + processing lead time) → it moves to a post-launch
>
> accelerator. Satellite stays only as a **free visual basemap** for reading morphology by eye. The
>
> binding constraint at launch is **editor throughput, not data processing** — so every design choice
>
> below optimises *correct km confirmed per editor-hour*.

---

2. How it works — the simple flow

Strip the math and the system is **five steps in order**. The first four run by machine, before anyone opens the tool; a person does only step 5, and only where step 4 says a person is needed.

1. **Guess from the road's shape.** Every stretch gets a *legal-default* speed from its morphology (expressway, divided rural, built-up…) via the Thông tư 38 rules. Free, instant, covers 100% of the network — but it's only the default, not a posted limit.
2. **Look for the real sign.** Pull street-view sign detections (Mapillary now; our own cameras/CV later) that override the default wherever a limit is actually posted.
3. **Clean and place each sign.** The same sign is seen many times with scattered GPS; collapse those into one point and work out which carriageway and direction it governs.
4. **Join it into a profile, and rate our confidence.** A road's speed holds over long *runs* and only jumps at *change-points* (a sign, a built-up boundary, a divided→single change). Build that step-profile and mark each run **confident / needs-a-look / no-data**.
5. **Human confirms with evidence, then it ships.** The editor works a ranked worklist, confirms the handful of uncertain change-points against the street-view photo, and the confirmed run is written back to OSM citing that evidence.

The whole point of steps 1–4 is to make step 5 small: a human *confirms a machine's suggestion against a photo*, instead of reading 28,000 km of road by hand. Fig. 1 (§6) shows this as a pipeline; everything after §4 is just *how each step is computed*.

---

3. What we're building, and how it grows

maxspeed is the **first attribute** on a general **road-attribute editor**, not a one-off. The same five-step flow — *prior → detect → localize → chain → human-confirm* — works for any attribute that is either **posted on a sign** or **readable from road shape**. So the platform grows along two axes, **attributes** and **evidence sources**, reusing the same machine.

Attribute	What estimates it	Evidence source(s)	When
maxspeed	TT38 morphology prior + posted sign	Mapillary signs · satellite morphology (basemap)	**Now — launch**
No-overtaking (cấm vượt)	posted sign	street-view CV	Next (same sign pipeline)
Residential enter/ exit (khu dân cư)	posted sign → built-up default	street-view CV	Next
Lane count	road shape	satellite/aerial CV · OSM	Phase 2 (CV)
Other prohibition signs	posted sign	own CV	Phase 4
Signals / intersection attrs	intersection geometry + signs	street-view + graph (intersection-based, hard)	Deferred

The **evidence stack** also deepens under all of them over time:

> street-view *lookup* (read by eye) → **Mapillary CV signs** (now) → **satellite-morphology CV**
>
> (purchased imagery, post-launch) → **our own dashcam fleet + retrained CV** (owns both the evidence
>
> and the retraining loop).

Each new attribute is **a new detector + a new sheet column**, not a new tool — the chain model, triage, editor sheet, evidence policy and write-back are shared. (Detection of signals and lanes stays *deferred*; the lanes QA view already ships in `traffic/lanes/`.)

---

4. Team & operating model

This is **not a solo effort, and not only an editing team** — it spans data, ML, tooling, operations and review. The operating loop is **machine proposes → MapOps confirms with evidence → Engineering publishes → Product monitors**, and each arm needs an owner. Listed as *roles*, some filled today, some to staff as the program grows:

Role	Owns	In the flow	When
Product / program lead	objective, scope, evidence policy, priorities	all	now
Geospatial data engineer	OSM/Mapillary/imagery ingestion, TT38 prior, suggestion store, dashboards	steps 1–4	now
Computer-vision engineer	sign/lane detectors, denoise+localize, own-CV retraining	steps 2–3	**to staff**
ML / inference engineer	chain model (SP5), confidence + triage thresholds (SP6)	step 4	now → soon
Tooling / frontend engineer	the editor (`traffic/lanes/`): sheet, map, profile, write-back	step 5	now
MapOps editors	confirm suggestions against evidence, flag conflicts	step 5	now (core) → **scale**
MapOps lead / QA	SOP, training, the revert/review gate, per-editor changeset attribution	step 5	now
Backend / platform engineer	Editor Spatial DB Server, OSM write-back + sync	step 5 (Phase 3)	later

How it scales. A small core hand-edits and **screen-records to build the SOP**; the SOP trains **paid-per-km part-time editors**; and those labelled confirmations become training data for an **AI-assisted auto-edit** path (always behind mandatory human review). Headcount, pay-per-km rate and the QA gate are product-owner decisions (§13). The **computer-vision engineer is the key to-staff role**: until it's filled the program runs on Mapillary's (European-trained, partial-VN) detector — which is exactly why own-CV is a post-launch accelerator, not a launch dependency.

> **The method, in depth (§5–§11).** The rest of this document is *how each step above is computed* —
>
> the estimation objective, the eight sub-problems, the chain model, and where satellite fits.
>
> Roadmap, decisions and sizing resume at §12.

5. The big problem, in one objective

We want to maximise the **correct, traffic-weighted kilometres** of maxspeed we can confirm within a fixed editor-hour budget before the launch date:

```
maximise   Σ_runs  km(run) · 1[ v̂(run) = v_true(run) ] · w_traffic(run)
subject to Σ editor-hours ≤ budget,   deadline = early September
```

A *run* is a maximal stretch of road over which the limit is constant (definition in §7). To pick `v̂(run)` we need a per-run estimate **with calibrated confidence** — so the inner problem is Bayesian estimation of the limit `v` on each segment `s`:

```
P(v_s | E)  ∝   P(v_s | morphology_s)        ← PRIOR     : Thông tư 38 legal default
              × Π_k P(e_k | v_s)              ← EVIDENCE  : independent observations (s
              × Π   P(v_s | v_neighbour)      ← COUPLING  : speed is spatially autocorr
```

Everything else in this plan is **how we compute each factor of that product, and how we turn the posterior into editor actions.** The two halves — *estimate the posterior* and *spend editor-hours optimally given the posterior* — are the two columns of the decomposition in §7.

> **One hygiene rule on the EVIDENCE term.** maxspeed.nl / MaxBit / most reference layers are
>
> *derived from OSM*, so they are **not** independent likelihood terms — counting them inflates
>
> confidence. Independent evidence = street-view signs, satellite morphology, field survey. Reference
>
> layers are glance-only corroboration (also a licensing line — §13).

---

6. System overview

The estimation runs **offline in pipelines**; the browser only renders suggestions and stages edits.

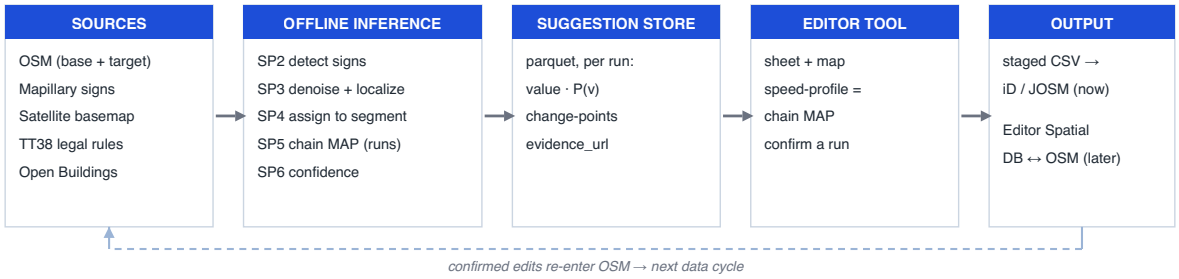

Fig. 1 — System data-flow. Detection and inference are offline; the editor consumes a precomputed suggestion store and stages edits.

7. Decomposition: one objective → eight sub-problems

Each factor of the posterior, plus the "spend editor-hours well" half, becomes a sub-problem we can build and test in isolation.

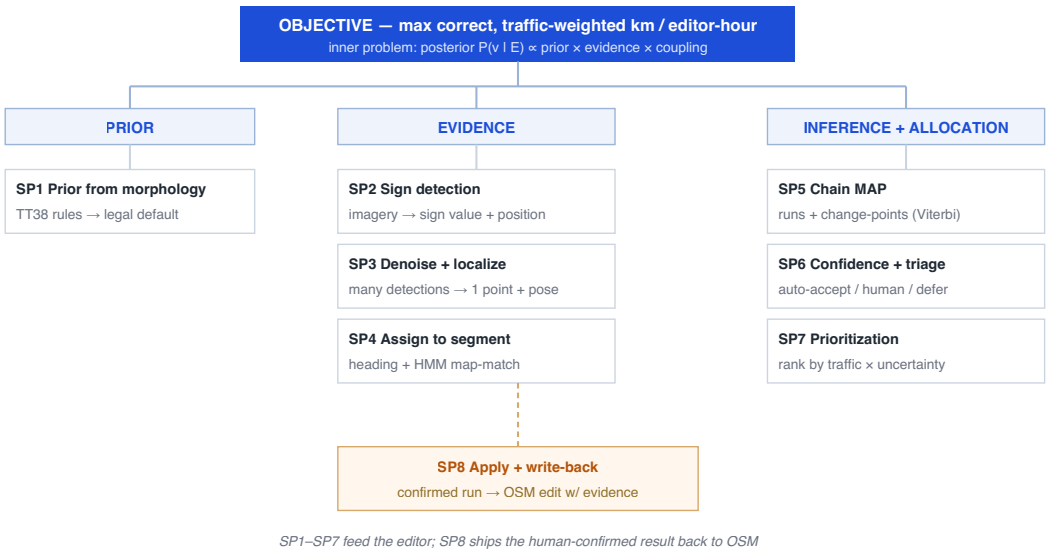

Fig. 2 — Problem decomposition. The objective splits into the three posterior factors (prior / evidence / coupling) plus the allocation half; eight sub-problems, each independently buildable.

The sub-problem ledger

#	Sub-problem	Input → Output	Method	Research / precedent	Launch-critical	Status
SP1	Prior from morphology	morphology tags → legal default `v`, `P(v|m)`	TT38 decision tree (rules)	Guth et al. 2020; TT38 38/2024	✅	**Done** — `legalUrban/Rural` on each `SheetRow`; subset morphology-% TBD
SP2	Sign detection	street imagery → sign value + position	CNN detector (Mapillary now; own CV later)	Ajmar 2019; RoadTagger 2020; Tusher 2024 (domain gap)	✅ (Mapillary)	Sidecar live; own CV post-launch
SP3	Denoise + localize	many noisy detections → 1 point + pose	context-aware confidence clustering	Kango 2024; Yang & Ai 2018	✅	**Not built** (new — §9)
SP4	Assign to segment	sign point + heading → `way_id`, direction	heading-augmented HMM map-match + side-of-road + `pairId`	Kango 2024; Newson & Krumm 2009; Liu 2020	✅	Heuristic (`pairId`) exists; HMM TBD
SP5	Chain MAP	per-segment prior + evidence → runs + change-points	linear-chain CRF/HMM, Viterbi (graph reduction)	RoadTagger 2020; Jepsen 2019/2022	✅ (heuristic)	Heuristic chain TBD; speed-profile UI done
SP6	Confidence + triage	posterior → auto-accept / human / defer	learned confidence + thresholds	Kango 2024 (auto-reviewer)	✅	Threshold policy TBD
SP7	Prioritization				✅	TBD

#	Sub-problem	Input → Output	Method	Research / precedent	Launch-critical	Status
		runs → ranked worklist	value-weighted info gain (traffic × uncertainty)	heuristic; Mapillary-density proxy		
SP8	Apply + write-back	confirmed run → OSM edit w/ evidence	staged CSV → iD/ JOSM now; Editor Spatial DB later	enterprise-architecture.md; Kango 2024	✗ (export now)	Export done; backend Phase 3

8. The chain model, made visual (SP5)

A road is **not** N independent segments — its attributes are **spatially autocorrelated** (the empirical premise of RoadTagger, He et al. 2020, and of Jepsen et al.'s OSM speed-limit work). The limit holds over long homogeneous **runs** and jumps only at **change-points**: a posted sign, a built-up enter/exit sign, a divided→single morphology change, a major junction. The estimate is the **Viterbi path** along the road; the editor's job collapses from "label thousands of segments" to "**confirm the handful of change-points**."

Fig. 3 — Speed as a chain. Confirm the change-points, auto-fill the runs between them. Each run's changeset cites the boundary sign — which is also how the law works (a sign governs until the next sign), so this defuses the revert/ban risk and collapses the work.

> **Model note.** A linear-chain CRF / HMM is the practical single-road *reduction* of the graph
>
> propagation in RoadTagger/Jepsen (a route is a degenerate graph). Ship the **heuristic chain**
>
> first (propagate confident signs, break at change-signs / morphology changes, else legal default);
>
> upgrade to a full CRF/graph model only if accuracy measurably needs it.

---

9. The new piece: denoise & localize, then assign (SP3 → SP4)

The hardest, least-built link. The same sign is detected many times with scattered geolocation error, so we must **cluster repeated detections into one representative point + pose before assignment** — confidence-weighted, context-aware on sign pose + detection angle + vehicle heading (Kango et al. 2024 beat fixed-distance clustering, esp. near intersections). Then map-match with **vehicle heading** (Mapillary `compass_angle`) + side-of-road + the existing `pairId` carriageway pairing — the classic HMM map-match (Newson & Krumm 2009) augmented with heading, which is what resolves carriageway / U-turn ambiguity (it lifted hard-case accuracy 26%→84% in Kango 2024). maxspeed is **way-based** (project to the nearest point on the matched way) — the tractable case; intersection-based signs (signals) are the hard, deferred class.

Pipeline output the tool consumes (one row per run): `[way_id, run_id, value, P(v), is_change_point, evidence_url, n_detections, cluster_spread, pose_angle]` — the last three are the strongest auto-accept-confidence features.

---

10. Spending editor-hours well (SP6 → SP7)

The posterior tells us **where a human is actually needed**. Three outcomes:

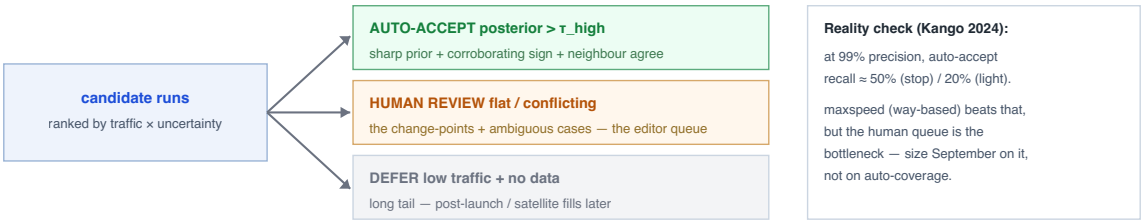

Fig. 4 — Triage. Auto-accept the confident, route the ambiguous to humans, defer the long tail. Editor-hours go only to the middle band.

Why the national roads are the low-hanging fruit — every posterior term favours them:

Term	On the big roads	Effect
Prior `P(v|morphology)`	morphology uniform + well-tagged (expressway→120, divided rural→90)	sharp prior, low entropy
Coupling	long consistent runs, few change-points / 100 km	one observation propagates over many km
Evidence cost	driven/captured most → dense Mapillary	cheap to confirm
Value	highest traffic	each correct edit worth the most

The same signal — **Mapillary density** — is both the value proxy and the evidence-cost proxy, and they coincide on exactly the roads we care about. *Caveat:* density is a **biased** traffic proxy (contributor behaviour, not AADT; skews to populated areas — Jepsen et al. 2022). Fine for ranking, not ground-truth volume.

---

11. Two-track evidence (where satellite fits)

TT38 keys speed on **morphology, not highway class**, which splits the evidence cleanly:

Track	Answers	Source	At launch
Satellite	morphology → the **legal-default prior** (đường đôi/đơn, lanes, built-up)	free basemap by eye now; purchased + CV later	**free basemap only**
Street-view	the **posted sign** that overrides the default (the change-points)	Mapillary now → own dashcam/CV	**primary evidence**

> **Compliance line (do not blur).** Satellite morphology → legal-default maxspeed is legitimate,
> scalable evidence. A *posted* (non-default) limit still needs the sign. Crossing this gets changesets reverted and accounts banned.

---

12. Roadmap

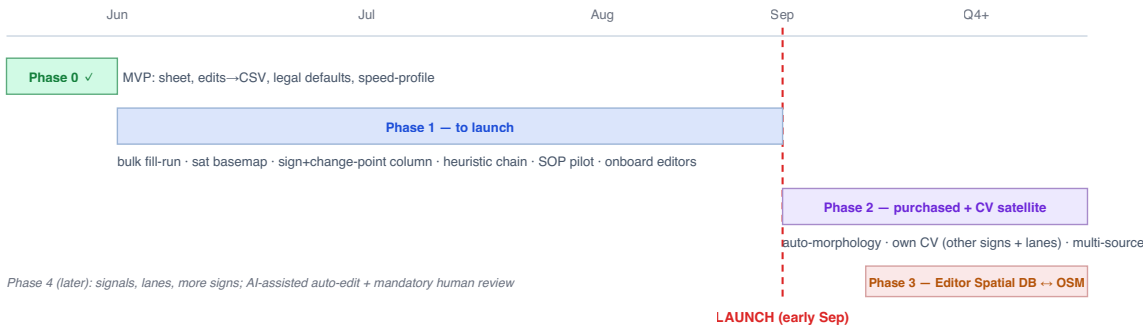

Fig. 5 — Roadmap. Phase 1 is the only thing on the launch critical path; purchased satellite and the write-back backend are deliberately post-launch.

- **Phase 0 — done.** Sheet, inline editing, staged edits → CSV, legal defaults, completeness, speed profile, map.
- **Phase 1 — to launch.** *Tooling:* bulk "fill whole run/road" action; free satellite basemap; Mapillary sign-suggestion + change-point column; enforce the evidence policy (§13). *Method:* ship the heuristic chain (SP5) + denoise/assign (SP3/SP4) → per-run confidence. *Ops:* measure km/editor/day on a 1–2 road SOP pilot; clear **all expressways** first (small km, near-100% fast win), then national highways easy→hard; onboard editors. *Tracking:* daily ranked-road dashboard (km done vs official) + **revert-rate** watch.
- **Phase 2 — post-launch.** Purchased + CV satellite auto-derives morphology; own CV for other signs + lanes; full multi-source aggregation + conflict flagging.

- **Phase 3 — backend.** Editor Spatial DB Server (local PBF / Postgres+PostGIS) syncing with OSM; write-back with evidence + per-editor changeset attribution + QA queue.
- **Phase 4 — scope.** Signals, lanes, more prohibition signs; AI-assisted auto-edit with mandatory human review.

---

13. Decisions the product owner owns

1. **Evidence policy (sets the speed ceiling).** Recommended **hybrid**: posted (non-default) limits require street-view evidence; legal defaults from visible morphology are tagged `source=survey/imagery` at human pace through editor accounts (not a bot import). Pure street-view-only is safe but coverage- and throughput-limited by Mapillary gaps.
2. **Imagery source & budget** — internal tile server vs commercial sub-meter; how much, where (coverage planner gives the buy-envelope). Drives lane-count feasibility. **Start procurement in parallel now** so it lands post-launch.
3. **Build vs buy CV** — own model (owns evidence + retraining loop) vs vendor.
4. **Write-back timing** — how long to stay export-first before funding the Editor Spatial DB.
5. **Editor ops** — headcount, pay-per-km rate, QA gate.

---

14. Throughput sizing (do this first)

Run `morphology_coverage.py` on the **national + expressway subset only**. The % carrying morphology tags = the share auto-fillable from the prior (cheap bulk); the rest needs an editor to read morphology off the free basemap. Then:

```
editor-days needed ≈ (28,000 km − auto-filled km) ÷ (editors × km/day)
```

Fill `km/day` from the SOP pilot (unmeasured; headcount/rates TBD). Because the method verifies **runs, not segments**, `km/day` should be far higher than naïve per-segment editing — that gap is the whole feasibility argument. **But don't over-size on auto-fill:** even production conflation holds 99% precision only at ~50%/20% recall (Kango 2024); the auto-accept path clears a *fraction*, the rest is human. Size September on the **human queue**.

---

15. Metrics & risks

Metrics. Coverage % of the ~28k km subset — measure the baseline **on that subset** (via `route_coverage.py` / `name_maxspeed_crosstab.py`), **not** the repo's 12.9% figure (that is over *all* tertiary+, 133,771 km — wrong denominator here) · km/editor/day · time-per-run · suggestion-accept rate · **revert rate (the quality gate)** · cost/km · traffic-weighted coverage.

Risks. Licensing contamination (Waze/Google) · evidence-less bulk edits → bans · Mapillary VN gaps (mitigated: big roads densest; satellite fills the prior) · **Mapillary sparsity ⇒ noisier localization than a controlled fleet** — clustering quality degrades with few traversals, so the Kango 2024 blueprint pays off fully only with our own dashcam fleet · **ML speed-limit models don't transfer across regions** — Jepsen et al. 2022 report cross-network accuracy collapsing to ~⅓, why we lean on legal prior + local evidence over a foreign-trained model · over-trust in a flat posterior (mitigated by triage) · backend conflict-resolution complexity (Phase 3).

---

16. Immediate next actions

1. **Run the morphology sizing** on the national + expressway subset → fixes the throughput target.
2. **Build the bulk "fill run to legal default" action + free satellite basemap** in the tool (SP1 → editor).
3. **Build SP3/SP4** (denoise + localize + heading-assign) and **SP5** heuristic chain → emit the per-run suggestion rows.
4. **Run the SOP pilot** (1–2 roads) to measure km/day and lock the evidence policy.
5. **Start satellite procurement in parallel** for the post-launch accelerator.

References

- He, S., Bastani, F., Jagwani, S., et al. (2020). RoadTagger: Robust road attribute inference with graph neural networks. *AAAI, 34*(07), 10965–10972. https://doi.org/10.1609/aaai.v34i07.6730
- Jepsen, T. S., Jensen, C. S., & Nielsen, T. D. (2019). Graph convolutional networks for road networks. *ACM SIGSPATIAL*, 460–463. https://doi.org/10.1145/3347146.3359094

- Jepsen, T. S., Jensen, C. S., & Nielsen, T. D. (2022). Relational fusion networks: Graph convolutional networks for road networks. *IEEE T-ITS, 23*(1), 418–429. https://doi.org/10.1109/tits.2020.3011799
- Kango, V., Eraqi, H. M., & Moustafa, M. (2024). *High precision map conflation of fleet sourced traffic signs.* Amazon. (context-aware clustering + heading-augmented HMM map-matching + auto-reviewer; 99.5% precision auto-ingest.)
- Newson, P., & Krumm, J. (2009). Hidden Markov map matching through noise and sparseness. *ACM SIGSPATIAL*, 336–343.
- Guth, J., Wursthorn, S., & Keller, S. (2020). Multi-parameter estimation of average speed in road networks using fuzzy control. *ISPRS IJGI, 9*(1), 55. https://doi.org/10.3390/ijgi9010055
- Yang, W., Ai, T., & Lu, W. (2018). A method for extracting road boundary information from crowdsourcing vehicle GPS trajectories. *Sensors, 18*(4), 1261. https://doi.org/10.3390/s18041261

Full scite-verified bibliography for the detection/evidence layers: `research/README.md`.